Topological Code-Graph Slicing and Contractive Lyapunov Verification for On-Device Gemma 4 SWE Agents

Token-Budgeted AST Graph Textualization and Inference-Time State-Space Backtracking on Single-Accelerator Hardware

Alessandro Massimiliano Gasparini (alexmaxi) — Google DeepMind: The Gemma 4 Developer Agent Paper Track
GitHub: `https://github.com/granbellllc-png/gemma-wayfinder-agent` | Kaggle GPU Notebook: `alexmaxi/gemma-4-tcgs-lyapunov-swe-agent-verification` | Apache 2.0

Abstract. Google DeepMind's open-weight **Gemma 4** models (E2B ~2.3B, E4B ~4.5B at 128K context; 12B, 26B A4B MoE ~3.8B active / 25.2B total, and 31B at 256K context; Gemma Team, 2026) enable private local software engineering (SWE) agents (Jimenez et al., 2024). Yet under tight local memory ceilings, agents confront two separable bottlenecks: (i) **graph-textualization loss** under flat/lexical retrieval (Zhang et al., 2023; cf. Markowitz et al., 2025) and (ii) **multi-turn limit-cycle thrashing** without structured state-space backtracking (Yao et al., 2023; cf. Markowitz et al., 2024). We study both layers orthogonally in **Wayfinder v3.1**: **Hierarchical Hop-Skeletonized Topological Code-Graph Slicing (TCGS)** and **State-Adaptive Discrete Contractive Lyapunov Verification (DCLV)** (Banach, 1922; Khalil, 2002) with a sub-millisecond Bellman-Ford `DPLL(QF_RDL)` invariant certificate (Nieuwenhuis et al., 2006). Across 45 targets in 5 Python stdlib packages (1,917 AST nodes, 5,997 edges), TCGS (k=2) achieves **54.7% 2-hop recall** vs. 18.7% for Okapi BM25 (+36.0 pp, Wilcoxon p = 7.62e-09) and 10.4% for PageRank RepoMap at **16.69× compression** (3,335 vs. 55,670 tokens). In factorial **live GPU evaluation on official `google/gemma-4` weights** (`Gemma 4 E2B-it` and `Gemma 4 E4B-it` on 2× Tesla T4, N = 20 7-file Python diagnostic repositories, 100 held-out unit tests, identical 6-call budget), Wayfinder achieves **20.0% vs. 5.0% BM25** on E2B-it and **60.0% vs. 40.0% BM25** on E4B-it (outperforming fixed-ratio contraction at 50.0%; exact McNemar p = 0.092285 vs. BM25, p = 0.375 vs. Fixed DCLV), with **0.0% cyclic thrashing** and **100.0% verified QF_RDL witnesses**.

1. Formal Control Architecture: State-Adaptive DCLV & DPLL(QF_RDL) SMT

Let observable repository state at turn t be $S_t = (F_t, T_t, U_t)$, where F_t maps tracked files to SHA-256 digests (baseline F_0), integer $T_t \geq 0$ counts failing unit tests, and integer $U_t \geq 0$ counts static AST/import errors. Targeting the zero-defect manifold $S^* = \{S : T = 0 \text{ and } U = 0\}$, we define the zero-oracle-leak Defect Lyapunov Potential on half-integer lattice $(1/2)\mathbb{N}_0$ (quantum $\Delta_{min} = 0.5$; Khalil, 2002; Banach, 1922):

$$V(S_t) = 1.5 \cdot T_t + 1.0 \cdot U_t + 0.5 \cdot \mathbb{I}(T_t > 0) \cdot |F_t \,\Delta\, F_0| \quad | \quad \lambda_{eff}(S_t) = \max(\lambda_0,\ 1 - 0.5 \,/\, V(S_t))$$

Theorem 1 (Finite-Step Lattice Convergence) & Theorem 2 (QF_RDL Witness Soundness). Under transactional rollback, SHA-256 state-hash Taboo H_t, and adaptive envelope $\lambda_{eff}(S_t)$, rejected proposals leave S_t strictly invariant, cyclic non-zero revisits equal 0.0%, and accepted trajectories reach $V = 0$ in $K_{adapt} \leq 2 \cdot V(S_0) + B_{neutral}$ steps (whereas fixed $\lambda_0 = 0.75$ over-rejects valid 1-file repairs on coupled multi-file bugs, e.g., V: 7.5 → 6.5 > 5.625). At every candidate step, `DifferenceLogicSMTVerifier` checks four difference-logic bounds $x_i - x_j \leq c_{ij}$ (Nieuwenhuis et al., 2006): $V(S') - V(S_t) \leq \Delta_{allowed}$, $0 - V(S') \leq 0$, $|F' \,\Delta\, F_0| - 0 \leq B_{files}$, and $\text{tokens}(S') - 0 \leq B_{token}$ via Bellman-Ford $O(|X| \cdot |E_c|)$, independently verifying witness π before commit.

2. Hierarchical Hop-Skeletonized Topological Code-Graph Slicing (TCGS)

Unlike iterative lexical retrieval (Zhang et al., 2023) or global centrality maps, `TopologicalCodeGraph` constructs a static directed AST call/import/inheritance manifold $G = (V, E)$ without executing untrusted repository code (hub damping at degree 24). Breadth-first slicing from focal symbol u_0 emits full executable AST bodies for hops $d \leq 1$ and compact typed interface skeletons (`signature + docstring`) for hop $d = 2$ within $B_{token} \leq 3{,}500$ tokens (<650 MB FP16 KV-cache on 16 GB Unified Memory workstations).

3. Experiment A: Real Python Standard-Library Slicing (1,917 AST Nodes, 5,997 Edges, N = 45)

Retrieval Strategy (≤3.5k Budget)	2-Hop Recall	Mean Tok	Compr.	26B A4B KV	Paired Δ vs. k=2 (95% CI, Wilcoxon p)
Full-Package Context (Unbounded)	100.0%	55,670	1.0×	10,298.95 MB	Baseline 100% (OOM >10 GB KV)
Global PageRank RepoMap	10.4%	3,497	15.92×	646.94 MB	+44.3 pp [37.3, 51.4], p=5.2e-09
Okapi BM25 Lexical	18.7%	3,494	15.93×	646.39 MB	+36.0 pp [30.5, 41.9], p=7.6e-09
Fixed 500-Token Chunks	22.1%	6,098	9.13×	1,128.13 MB	+32.7 pp [26.1, 39.5], p=1.4e-08
TCGS k=1 (1-Hop Only)	21.4%	2,605	21.37×	481.93 MB	+33.4 pp [27.4, 39.6], p=5.2e-09
TCGS k=2 Hop-Skeletonized (Ours)	**54.7%**	**3,335**	**16.69×**	**616.98 MB**	**Reference Optimum (<1.8 GB M4 Budget)**
TCGS k=3 Hop-Skeletonized	55.1%	3,454	16.12×	638.99 MB	+0.4 pp diminishing return

4. Experiment B: Live On-GPU Gemma 4 Neural Repair Benchmark (N = 20 Multi-File Repos, 100 Unit Tests)

We evaluate official google/gemma-4 weights (Gemma 4 E2B-it and Gemma 4 E4B-it; Transformers 5.20.2, PyTorch 2.11.0+cu130 on 2× Tesla T4) across 20 seven-file Python repositories (5 domains, held-out 5-test suites, 450-token slice budget, identical 6-LLM-call ceiling):

Live On-GPU Model & Agent Harness Configuration	Resolved	Calls	Accept Tok	Billed Tok	Rollbacks	Thrash
Gemma 4 E2B-it + Okapi BM25 (Ungoverned ReAct)	1/20 (5.0%)	6.00	5,442	5,442	0	50.0%
Gemma 4 E2B-it + TCGS k=2 (Ungoverned ReAct)	4/20 (20.0%)	5.35	4,777	4,777	0	70.0%
Gemma 4 E2B-it + TCGS k=2 + Fixed DCLV (lambda_0=0.75)	3/20 (15.0%)	5.55	721	5,616	96	0.0%
Gemma 4 E2B-it + Wayfinder Full (TCGS k=2 + Adaptive DCLV + QF_RDL)	**4/20 (20.0%)**	**5.45**	**1,400**	**5,381**	**79**	**0.0%**

Live On-GPU Model & Agent Harness Configuration	Resolved	Calls	Accept Tok	Billed Tok	Rollbacks	Thrash
Gemma 4 E4B-it + Okapi BM25 (Ungoverned ReAct)	8/20 (40.0%)	4.90	4,500	4,500	0	40.0%
Gemma 4 E4B-it + TCGS k=2 (Ungoverned ReAct)	12/20 (60.0%)	3.70	3,307	3,307	0	35.0%
Gemma 4 E4B-it + TCGS k=2 + Fixed DCLV (lambda_0=0.75)	10/20 (50.0%)	4.15	1,290	3,966	55	0.0%
Gemma 4 E4B-it + Wayfinder Full (TCGS k=2 + Adaptive DCLV + QF_RDL)	**12/20 (60.0%)**	**3.70**	**2,207**	**3,440**	**25**	**0.0%**

5. Experiment C (Deterministic QF_RDL Stress Suite) & Appendix Synthetic Horizon Sensitivity

Experiment C (Scripted Adversarial Patch Stress Suite, N = 15 3-File Repos, 60 Tests): Under scripted regression/oscillation streams, unguided execution resolves 0/15 (0.0%, 100% cyclic thrash), whereas Wayfinder resolves **15/15 (100.0%)** with **36** transactional rollbacks and **30/30 (100.0%)** independently verified Bellman-Ford `QF_RDL` SAT witnesses.

Appendix Synthetic Parametric Markov Simulation (Explicitly Simulated Horizon Sweep, N = 250, seed=20261110): To illustrate asymptotic horizon sensitivity beyond 6 turns when parameterized by Experiment A recall, fixed-λ_0 sweep yields [λ_0=0.50: 21.2% (K≤5) | λ_0=0.65: 38.8% (K≤7) | λ_0=0.75: 58.4% (K≤10) | λ_0=0.85: 60.0% (K≤18) | λ_0=0.95: 66.0% (K≤55)], while adaptive DCLV reaches 66.8% vs. 53.6% fixed λ_0=0.75 (0.0% thrash).

Scope, Governance & Reproducibility: Exp B's 20 7-file suites are controlled diagnostic benchmarks isolating 2-hop contract coupling; static `ast.parse` executes zero untrusted code and omits dynamic reflection (`tree-sitter` extends TCGS to Rust/Go/TS); tests execute in isolated ephemeral sandboxes on unmodified `google/gemma-4` checkpoints under Apache 2.0 (`GPU SHA-256: acf9de0b44c37be30448068d...`).

References

[1] Gemma Team, Google DeepMind (2026). *Gemma 4 Technical Report*. arXiv:2607.02770.

[2] Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. (2024). *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* ICLR 2024.

[3] Markowitz, E., Ramakrishna, A., Dhamala, J., Mehrabi, N., Peris, C., Gupta, R., Chang, K.-W., & Galstyan, A. (2024). *Tree-of-Traversals: A Zero-Shot Reasoning Algorithm for Augmenting Black-box Language Models with Knowledge Graphs*. ACL 2024, 12302–12319.

[4] Markowitz, E., Galiya, K., Ver Steeg, G., & Galstyan, A. G. (2025). *KG-LLM-Bench: A Scalable Benchmark for Evaluating LLM Reasoning on Textualized Knowledge Graphs*. arXiv:2504.07087.

[5] Zhang, F., Chen, B., Zhang, Y., Keung, J., Liu, J., Zan, D., Mao, Y., Lou, J.-G., & Chen, W. (2023). *RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation*. EMNLP 2023.

[6] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR 2023.

[7] Banach, S. (1922). *Sur les opérations dans les ensembles abstraits et leur application aux équations intégrales*. Fundamenta Mathematicae, 3(1), 133–181.

[8] Khalil, H. K. (2002). *Nonlinear Systems* (3rd ed.). Prentice Hall.

[9] Nieuwenhuis, R., Oliveras, A., & Tinelli, C. (2006). *Solving SAT and SAT Modulo Theories: From an Abstract Davis–Putnam–Logemann–Loveland Procedure to DPLL(T)*. Journal of the ACM, 53(6), 937–977.